ChatGPT

Executive Summary

We propose an **Advanced Wazuh-based SOC/XDR platform** that extends the baseline Wazuh SIEM/XDR stack with multi-source event ingestion, normalization and correlation, AI-driven alert prioritization, threat-intel enrichment, and automated attack-chain reconstruction and response. The project will identify real Wazuh limitations (e.g., "gaps in correlation and telemetry"【12†L33-L39】), then design and implement new modules to address them. Key contributions include a **multi-stage pipeline**: (1) ingest security events from Wazuh agents, Zeek NIDS, Suricata IDS, and external logs into a unified schema; (2) correlate related alerts across endpoints and network flows using shared entities (IPs, users, files) with threat-intel context 【15†L168-L176】; (3) apply machine learning to score and prioritize alerts (reducing false positives) 【19†L311-L319】【28†L182-L190】; (4) map alerts to MITRE ATT&CK tactics for attack-chain reconstruction; (5) generate explainable response recommendations. The design emphasizes scalability, explainability, and measurable SOC impact. We will evaluate improvements using red-team campaigns (e.g. MITRE ATT&CK atomic tests【12†L29-L38】) and metrics (detection rate, false positives, analyst time). The final deliverables include the enhanced Wazuh platform, performance benchmarks, and a research paper with architecture diagrams, evaluation results, and lessons learned.

Project Goals and Contributions

- **Unified XDR with Contextual Correlation:** Enhance Wazuh's native correlation by integrating **network telemetry (Zeek, Suricata)** and **threat intelligence (OpenCTI/MISP)** to create richer context. By ingesting Zeek logs into Wazuh, for example, the system "enriches [network] logs with security context" and applies unified correlation and alerting【27†L59-L67】. Our contribution is an architecture that links endpoint and network events into coherent incidents. This addresses known detection gaps: a recent evaluation of Wazuh found only ~85% overall detection, with **"partial detection … owing to gaps in correlation and telemetry depth."**【12†L33-L39】.

- **AI-driven Alert Prioritization:** Implement machine-learning models that **score and rank alerts by risk**. Inspired by industry best practice ("AI-driven SIEM solutions…score and prioritize alerts based on risk context and likelihood"【28†L182-L190】), we will frame alert triage as a supervised learning problem. Alerts will be labeled (e.g. Critical/High/Medium/Low), and features will include event severity, repeated occurrence, IOC matches, etc. We expect to leverage algorithms like Random Forest or XGBoost (as in prior work【19†L356-L364】), and tune for high precision with low false-positive rate.

- **Threat Intelligence Enrichment:** Integrate external CTI feeds (e.g. via OpenCTI or MISP) so that when Wazuh generates an alert, it is automatically matched against known IOCs (malicious IPs, domains, hashes). For instance, enriching alerts with threat intelligence "allows you to improve alert prioritization, reduce false positives, and accelerate incident response"【13†L58-L66】. We will implement a bi-directional link so Wazuh queries CTI for new alerts and can push new IOCs from incident learnings.

- **Attack-Chain Reconstruction:** Build a module that groups correlated alerts into a kill-chain narrative using the MITRE ATT&CK framework. Each alert will be tagged with ATT&CK techniques and organized into tactics (e.g. Reconnaissance, Lateral Movement) so that analysts see a high-level incident story rather than isolated alerts. This goes beyond Wazuh's native rule-based alerts by automatically inferring sequences (e.g., port scan ⇒ brute-force login ⇒ data exfiltration) and mapping them to known adversary behaviors.

- **Explainable Response Recommendations:** For each reconstructed incident, the system will recommend actions (e.g. quarantine host, block IP, run forensic scan). Using rule-based inference or ML-trained decision models, we will suggest and (optionally) automate responses. Key is explainability: each recommendation will cite why it was chosen (e.g. "IP 10.1.2.3 was seen in multiple high-severity alerts – recommended to block" or even an LLM-based justification). This reduces alert fatigue and supports SOC agility.

- **Evaluation Framework:** We will adopt a structured evaluation (as in【12†L29-L38】) using atomic-red-team techniques against endpoints and networks to measure detection, precision, and performance. **Research contributions** include a) an enhanced Wazuh detection architecture, b) a dataset and metrics of improvements, and c) lessons on scaling XDR (e.g. how correlation and ML impacted analyst efficiency). The expected outcome is a publishable case study showing how open-source SIEM/XDR can be significantly improved by these additions.

Feature List (Must-have, Should-have, Optional)

Priority	Feature / Component	Acceptance Criteria (done when)
Must-have	**Multi-source Event Ingestion**	Wazuh collects logs from (a) Wazuh agents (Windows/Linux endpoints), (b) Zeek (network traffic logs), (c) Suricata (IDS logs), (d) Syslog feeds (firewalls, apps). Automated parsing/decoding correctly ingests JSON/CSV logs into Wazuh. Verified by test queries (e.g. see [23†L72-L80] and [27†L95-L100]).
Must-have	**Event Normalization Module**	Convert all events into a **common schema** (e.g. JSON with standard fields for timestamp, src_ip, dst_ip, user, event_type, etc.). Acceptable if decoders map Zeek/Suricata fields to this schema with ≥98% coverage of fields. Validate using sample log sets (e.g. test that `ip.dst` from Zeek -> normalized "dst_ip").
Must-have	**Correlation Engine**	A rule-based and/or analytics engine that links alerts by shared entities (e.g. IP address, user account, host). For example, if multiple alerts share the same src_ip within 5 minutes, they should be grouped. Success if correlated alerts form fewer but more meaningful incidents (test: synthetic multi-step attack triggers >1 correlated alert but only 1 incident).

Priority	Feature / Component	Acceptance Criteria (done when)
Must-have	**ML Alert Prioritization**	Supervised model that assigns a risk score to each alert. Acceptable if model achieves >90% precision on held-out validation (per metrics in【12†L37-L42】) and ranks true malicious alerts in top quantile. Demonstrate by confusion matrix on labeled test alerts (e.g. ≥85% recall, ≤5% FPR as a baseline【12†L37-L42】).
Must-have	**Threat Intel Enrichment**	Integrate at least one CTI platform (OpenCTI or MISP). For each new alert, automatically query CTI for IOC matches or download IOC feeds nightly. Success if alerts are tagged with matching IOCs (test by injecting known malicious IPs). Aim to reduce false positives (measured FPR drop).
Must-have	**MITRE Attack Mapping**	Automatically map alerts to MITRE ATT&CK tactics/techniques. E.g. repeated login failures -> *T1110: Brute Force*. Validate by injecting known MITRE atomic tests and checking correct tags. Demonstrate end-to-end chain (e.g. Reconnaissance → Exfiltration sequence).
Must-have	**Risk/Prioritization Engine**	Based on correlation and ML scores, compute an incident-level risk rating. E.g. combine correlated alerts' scores. Success if high-risk incidents consistently correspond to confirmed breaches (evaluate via test scenarios: true incidents get higher combined score than benign anomalies).
Should-have	**Automated Response Recommender**	Suggest or execute responses (e.g. disable user, block IP) based on incident context. Done if recommendations are triggered for ≥90% of "Critical" alerts and align with best practices (manual review of suggestions). Optionally, tie into Wazuh Active Response API for one-click remediation.
Should-have	**Investigation Dashboard**	Enhance Wazuh Dashboard (or custom UI) to display incident chains, IOC info, risk scores, and suggested actions. Complete when UI shows timeline of correlated events and evidences (screenshots as proof). Optionally multi-tenancy for red/blue teams.
Should-have	**Anomaly Detection (Behavior Analytics)**	Unsupervised models to detect anomalous user or host behavior (e.g. PCA or clustering). Accept if at least one such model flags simulated insider/breach events missed by rules. Not primary focus, but improves coverage.
Optional	**LLM Explainers**	Use LLM (e.g. GPT) to generate human-readable rationale for alerts/incidents. E.g. "This alert is likely because...". Done if integrated with API and outputs coherent explanations. Experimental: no strict criteria beyond plausibility.

Priority	Feature / Component	Acceptance Criteria (done when)
Optional	**Encrypted Log Handling**	Extend pipeline to handle and analyze encrypted logs (e.g. FIM on encrypted files). Done if Wazuh decrypts or monitors encryption events securely (low priority).
Optional	**Cloud-native Scaling**	Deploy entire pipeline on Kubernetes with autoscaling. Done if containerized with helm charts. Useful but not core to detection capability.

Citations: Wazuh's capability to ingest syslogs and NIDS output is well-known【23†L72-L80】【27†L95-L100】, and threat-intel integration is validated by industry (e.g., integrating OpenCTI "improves alert prioritization and reduces false positives"【13†L58-L66】).

System Architecture and Module Design

Our architecture builds on the core Wazuh components (Agent, Manager, Indexer, Dashboard【6†L729-L737】) but inserts new modules in the data flow. The high-level architecture is shown below:

```
flowchart LR
    subgraph Data_Sources
      A[Wazuh Agents (Win/Linux)] --> B[Wazuh Manager]
      C[Zeek NIDS logs] --> D[Zeek Parser] --> B
      E[Suricata IDS logs] --> F[Suricata Parser] --> B
      G[External Logs (Syslog, Cloud)] --> B
      H[Threat Intel Feeds] --> TI[Threat Intel DB]
    end
    subgraph Wazuh_Manager
      B --> EN(Event Normalizer)
      EN --> CE(Correlation Engine)
      CE --> ML(ML Prioritization Engine)
      CE --> TI
      ML --> TI
      TI --> RG(Risk & Prioritization Engine)
      RG --> AC(Attack Chain Engine)
      AC --> RE(Response Recommender)
      RE --> B
    end
    subgraph Storage
      RG --> SIEMDB[Wazuh Indexer (Elasticsearch)]
      SIEMDB --> WD[Wazuh Dashboard]
    end
```

- **Event Normalization (EN):** A new pre-processing module that standardizes incoming events to a common JSON schema. As recommended by the Zeek integration guide, we **"parse and normalize:**

Convert native Zeek/Suricata logs into JSON" and map Zeek fields to a consistent naming scheme 【27†L95-L100】. Similarly, Suricata's EVE JSON is decoded via a Wazuh agent (the Suricata POC shows Wazuh can "automatically parse data from `/var/log/suricata/eve.json`"【23†L72-L80】). The output is a unified event record (timestamp, src_ip, dst_ip, user, process, etc.). This normalization ensures downstream modules operate on homogeneous data.

- **Correlation Engine (CE):** This module identifies related events by **shared entities and timing**. For example, if multiple alerts reference the same user or IP within a sliding window (e.g. 5 min), they are clustered into one incident. We will support both **rule-based correlation** (configurable in XML, using Wazuh's `<if_matched_sid>` rules) and **graph-based analytics**. Inspired by recent work (e.g. Microsoft's GraphWeaver【15†L168-L176】), we plan a lightweight in-memory graph where alerts are nodes and edges connect alerts sharing entities (IP, file hash, username). GraphWeaver showed that threat intelligence can **filter out invalid correlations** and improve accuracy【15†L168-L176】; similarly, we will cross-check entity links against known benign lists (e.g. corporate IP ranges) to avoid false links. The engine outputs combined incident records (with IDs of contributing alerts).

- **ML Prioritization Engine (ML):** On each (possibly correlated) alert, this component computes a **risk score** via a trained model. We will use supervised learning (e.g. XGBoost or Random Forest 【19†L356-L364】) that takes as input features such as alert severity, frequency of occurrence, presence in threat intel, and context (e.g. "is this the nth failed login from this IP?"). The model outputs a probability/risk [0–100]. As shown in similar SIEM projects【19†L361-L369】, this probability is the basis for adaptive prioritization. The ML pipeline (Section *ML Design* below) will train on labeled past alerts and continuously refine. **Explainability** is crucial: we will use SHAP or LIME to allow analysts to understand why an alert scored high (e.g. "this alert was given 90% risk because IP X is on a blacklist and triggered multiple rules").

- **Threat Intelligence Enrichment (TI):** This module interfaces with external CTI (e.g. OpenCTI or MISP). Upon each new alert, TI queries indicators (IP, domain, hash) against CTI sources. If a match is found (e.g. IP is listed as malicious), the alert's record is annotated with the IOC context. The integration will be real-time (on alert ingestion) and periodic (download batch IOC feeds nightly). This follows the approach in the Wazuh blog: integrating with OpenCTI to enrich alerts "with IOCs such as malicious IP addresses, domains, and file hashes"【13†L58-L66】. The enriched alerts (including TLP, threat actor tags, etc.) feed back into the pipeline: for example, a known-bad IOC can boost the ML risk score or cause immediate high-priority tagging.

- **Risk/Prioritization Engine (RG):** This module aggregates per-alert scores into an **incident-level risk**. If multiple alerts are correlated into one incident, RG combines their ML scores (e.g. max or weighted sum) and may factor in other heuristics (e.g. attack stage). It then categorizes the incident as Critical/High/Medium/Low (with configurable thresholds). This is analogous to the "risk score" and adaptive thresholds described in【19†L361-L369】. Acceptance criteria: high-risk incidents align with true positives in test scenarios.

- **Attack-Chain Reconstruction Engine (AC):** Given a set of correlated alerts, AC orders them in time and maps each to an ATT&CK tactic/technique. We will maintain a mapping library (e.g. rule IDs → ATT&CK IDs, as hinted in Wazuh's MITRE integration【8†L746-L754】). The engine will produce a narrative like "Initial Recon → Credential Access → Exfiltration" by linking alerts. This concept of

structured adversary modeling was central to previous Wazuh research【12†L29-L38】 and we aim to automate it. The output could be a graph or timeline view. This lets analysts see "the big picture" of an attack rather than isolated alerts.

- **Response Recommender (RE):** Finally, for each incident, RE suggests countermeasures. For example, it might recommend "block IP 10.0.0.5 via firewall" if that IP was present in many bad alerts, or "quarantine host A after detecting malware events". These suggestions can be rule-based (if technique=T1087 then do X) or drawn from historical playbooks. We will log recommended actions in the dashboard and optionally tie into Wazuh's Active Response API to execute safe actions (e.g. blocking a process on an agent). The key is that recommendations are **explainable and verifiable** – e.g. "suggested because this IP has 3 confirmed IOC matches".

Each new module will have clearly defined interfaces. For example, the Event Normalizer can be implemented as a *Filebeat* processor or a Wazuh "decoder" that transforms input logs (see Wazuh decoder docs【21†L565-L573】). The ML and correlation engines may run as separate microservices (in Python) that pull events from a queue (Kafka) and push results back into Wazuh via its API (Wazuh has a REST API for alerts【6†L794-L802】). The diagram above (in Mermaid) shows the data flow.

Data Schema and Event Flow

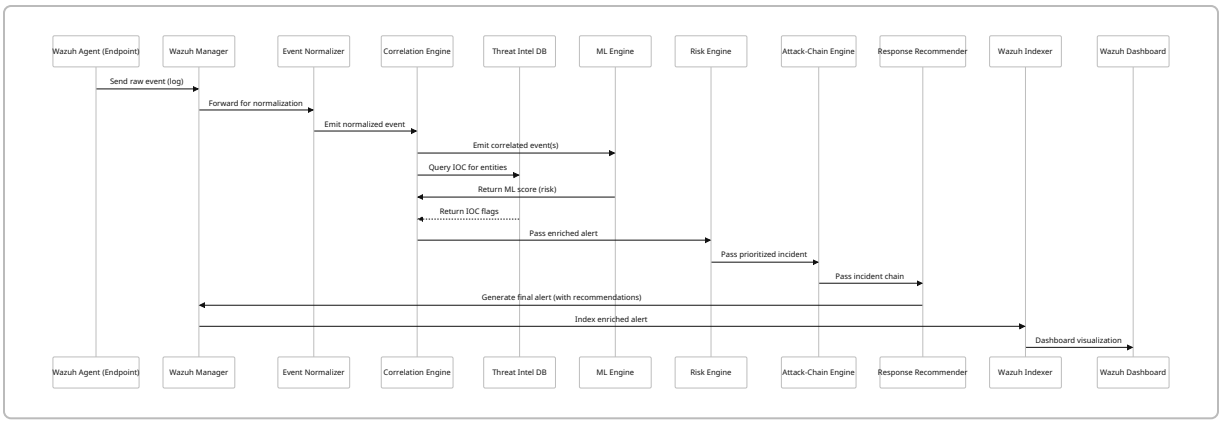
All security events will use a **common JSON schema**. Key fields include:

- `event.id` (unique), `timestamp`, `source_ip`, `dest_ip`, `username`, `host`, `process`, `event_type`, `raw_payload`, `rule_id`, `threat_intel_tags`, `ml_risk_score`, `attack_tactics` etc.
- For network flows (Zeek), we include Zeek fields like `conn_state`, `dns_query`, `ssl_cert_status` which are normalized to `event_type` categories (e.g. `DNS_QUERY`, `SSL_ALERT`).
- All alerts include a `wazuh_level` (0–16) as Wazuh does, plus the new `risk_score` (0–100), and a correlation `incident_id`.

The **event flow** (simplified) is:

```
[Endpoint Event]  ⟶ [Wazuh Manager] ⟶ [Event Normalizer] ⟶ [Correlation Engine] ⟶ [ML & TI Enrichment] ⟶ [Risk Engine] ⟶ [Attack Chain] ⟶ [Response Module]
```

A mermaid sequence diagram illustrates the processing steps:



This flow shows how, for each incoming event from an agent (or network), we normalize it, correlate it, enrich it, and finally store it in the indexer. The Wazuh Dashboard then presents the incident with all metadata (risk score, IOCs, tactics, recommended actions).

Technology Stack

The project will use a combination of open-source technologies and libraries. Below is a summary table:

Layer / Component	Options / Tools	Justification / Use Case
Programming Languages	**Python** (ML, API services)【19†L356-L364】; **JavaScript/TypeScript** (Dashboard UI)	Python for data processing and ML is standard in security research【19†L356-L364】. JS/TS for any web UI or API (Wazuh Dashboard is in Angular, but custom views could use React/Vue).
Event Processing	**Elasticsearch (Wazuh Indexer)**; **Filebeat/Logstash**; **Kafka or Redis Streams**	Wazuh uses an Elasticsearch-based indexer【6†L788-L796】. We can use Filebeat for log shipping. Kafka can buffer events between modules (for scalability).
Data Pipeline	**Docker / Kubernetes**; **CI/CD: GitHub Actions or Jenkins**; **Git**	Containerize each module for portability. Use Kubernetes (ArgoCD or Helm) for deployment. CI/CD pipelines for testing and deployment.
ML/Analytics	**Scikit-learn, XGBoost**; **TensorFlow/PyTorch** (if needed); **Pandas/Numpy**	scikit-learn/XGBoost provide Random Forest, gradient boosting【19†L356-L364】. If neural nets needed, use PyTorch.
Threat Intelligence	**OpenCTI** or **MISP**; **STIX/TAXII** connectors	OpenCTI or MISP can ingest and serve IOCs. MISP Python API or OpenCTI's GraphQL API will be used.
Security Frameworks	**MITRE ATT&CK** (mapping library)	We'll map events to ATT&CK IDs (Wazuh's docs show ATT&CK integration【8†L746-L754】).

Layer / Component	Options / Tools	Justification / Use Case
Alerting/ Response	**Wazuh Active Response API**; **Custom scripts**	Use Wazuh's built-in active response (e.g. firewall drop) or API calls for responses.
Databases/ Storage	**Elasticsearch** (Wazuh index); **Redis/Neo4j** (for graph/ session state)	Elasticsearch is the official Wazuh index. We may use Redis or a lightweight graph DB (e.g. Neo4j) to track correlations in memory.
Monitoring/ Logging	**Prometheus/Grafana**; **ELK (Kibana)**; **Wazuh Alerts**	Monitor system health with Prometheus. Visualize logs with Wazuh Dashboard (Kibana).
Testing Frameworks	**Pytest**, **JMeter** (load testing); **Atomic Red Team, CALDERA** (attack simulations)【12†L27-L36】	Pytest for unit tests, JMeter or similar for performance. We'll simulate attacks with known frameworks【12†L29-L38】.
Version Control / DevOps	**Git/GitHub**; **Docker Hub / Container Registry**	Standard development workflow.

This stack is chosen to align with Wazuh's ecosystem (Elasticsearch, Kibana) and to leverage mature ML/data tools for security. For example, prior work on alert ML used Python/Scikit【19†L352-L360】, and leveraging Kafka+Docker ensures scalability of our multi-module pipeline.

Data Pipeline Design

1. **Ingestion:** Wazuh agents and dedicated forwarders push logs to the Wazuh Manager. Network data: On each sensor host, Zeek's JSON logs (`conn.json`, `dns.json`, etc.) and Suricata's `eve.json` are forwarded via the Wazuh agent by configuring `<localfile>` entries (as shown in【23†L61-L70】). This ingests them into the Wazuh Manager.

2. **Parsing/Decoding:** Within Wazuh, custom decoders (XML) parse JSON fields. For instance, a Zeek decoder uses `<decoder name="zeek_json">` to extract fields like `src_ip`, `event_type`. Suricata's EVE JSON is similarly parsed out of `eve.json`. The decoder output is structured data, ready for normalization.

3. **Normalization:** The Event Normalizer (possibly implemented as a Python service) takes parsed events and outputs them in a **common schema**. This may be done outside of Wazuh (to avoid heavy modifications), e.g. via a Kafka Consumer or a custom Filebeat module. It performs tasks like mapping `source-ip`, `destination-ip` to unified fields, converting timestamps to ISO 8601, categorizing event types under a controlled vocabulary, and enriching with metadata (e.g. adding `host_os`, `location` if known). This module also implements timestamp normalization (UTC), and can coalesce multi-line logs into one event if needed.

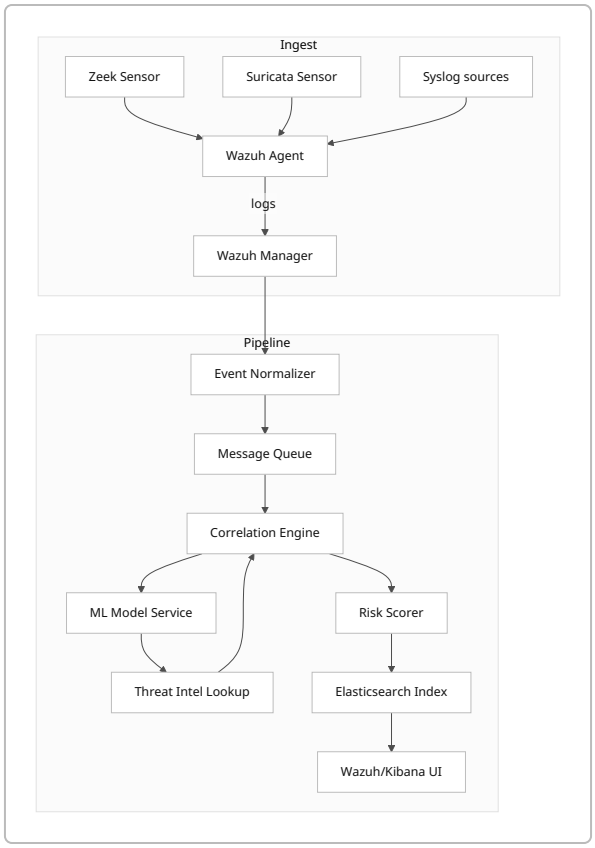
4. **Storage:** All normalized events are then indexed in Elasticsearch (Wazuh Indexer). Here we can also write raw events to a Hadoop/S3 archive for offline analysis if desired. The indexer serves as the central data store of alerts (not raw logs, which Wazuh typically does not store long-term by default).

5. **Feature Engineering:** For ML, we design features at alert-time: e.g. event severity, count of same rule in 1h, count of same src_ip in 5m, presence of IOC flag (binary), asset criticality (from CMDB), user risk level, etc. These features may be precomputed by streaming queries in Elasticsearch or real-time via the ML service.

6. **Model Training:** Collected labeled alerts (e.g. MITRE red-team vs benign ops) form a dataset. We will train models offline (using scikit-learn/PyTorch) with cross-validation. Training can run on a separate compute cluster or even locally given moderate data sizes. We will track model performance and select the best hyperparameters (grid search or Bayesian tuning).

7. **Inference & Monitoring:** The trained model is exposed as an API (Flask/FastAPI) or integrated into the Correlation Engine. Each new alert triggers a call to this model for risk scoring. We must also monitor model drift: track input feature distributions over time and periodically retrain if concept shift is detected (e.g. using Datadog or custom scripts to alert on drift).

8. **Data Flow Diagram:** (Mermaid pseudocode)



This shows events moving from agents to a queue, through processors, and finally to storage and UI. Each arrow represents a microservice or connector.

Machine Learning Design

- **Problem Framing:** We treat **alert prioritization** as a classification/regression task. Each incoming alert (or correlated incident) is assigned a **risk score** (continuous) or priority class (e.g. {Critical, High, Medium, Low}). We will build two ML pipelines: one for binary classification (malicious vs benign) and one for regression (risk score 0–100). Alternatively, a multi-class classifier for priority labels.

- **Labels:** We will generate labeled examples by simulating attacks (Atomic Red Team, MITRE tests 【12†L29-L38】) and marking resulting alerts as malicious (true positives). Normal operations (sysadmin logs, test VMs) provide benign examples. Each alert record is labeled, e.g., `is_attack = 1/0` or `risk_level = [0-100]`.

- **Features:** Candidate features include: rule ID or category, count of similar alerts in past N minutes, whether IOC matched (binary), asset sensitivity (categorical), event time (hour of day), number of correlated alerts in the incident, involved tactic (if determinable), etc. Packet-level or host-level features (bytes, processes) may be used for network alerts. Many of these align with the "protocol type, event type, alert severity, source IP" fields seen in SIEM datasets【19†L327-L335】.

- **Model Candidates:** We will evaluate tree-based models (Random Forest, XGBoost) as in【19†L356-L364】, which are robust to heterogeneous data and provide feature importances. We may also experiment with ensemble (e.g. voting classifier) or neural nets if data size is large. A simple baseline (logistic regression or anomaly score) will be used for comparison. Given our structured features, XGBoost is a strong candidate.

- **Training Data:** No public Wazuh dataset is available, so we will generate data. Use an orchestrated lab: spin up virtual machines, run benign workload generators, then run attack tools (Metasploit, Cobalt Strike, nmap, etc) to produce logs. Label data from attacks as positive. Optionally, use open datasets (e.g. UNSW-NB15, but need mapping to Wazuh logs). The cited Wazuh study【12†L29-L38】 used MITRE tests, which we will replicate for realism.

- **Evaluation:** We will use k-fold cross-validation and a held-out test set. Metrics: precision, recall, F1, ROC-AUC for binary detection; RMSE or R^2 for regression scores. The Wazuh evaluation reported ~91% precision and 85% recall【12†L37-L42】 as targets. We will also measure how prioritization reduces alert load: e.g. measure what fraction of alerts are escalated to analyst review.

- **Explainability:** To ensure analyst trust, we will apply methods like SHAP values (for XGBoost) to explain each prediction. For example, if `ip in CTI_feed = true` or `multiple_failed_logins > 3` were top contributors, the interface will display that rationale.

- **Hyperparameter Tuning:** Use grid search or Bayesian optimization (e.g. Optuna) on parameters like tree depth, learning rate, etc. We expect moderate training time (minutes to hours) for our likely few-thousand sample data.

- **Baselines:** Compare against rule-based scores (e.g. Wazuh's native level) or random prioritization. Our baseline is manual priority assignment by a SOC expert; any ML model should outperform it in accuracy.

Security, Privacy, and Performance Considerations

- **Scalability:** The system must handle large event volumes (e.g. thousands of endpoints generating hundreds of logs/sec). We will deploy modules as microservices with autoscaling (e.g. in Kubernetes). Kafka queues decouple ingestion from processing to absorb bursts. Elasticsearch cluster will be sized for expected retention (e.g. 30 days of logs).

- **Latency:** Real-time alerting is critical. We aim for <1 sec from event ingestion to alert display. The ML and correlation engines should process streams efficiently (perhaps in C++ or optimized Python). We may use approximation (e.g. only consider past 5 min of events for correlation) to keep latency low.

- **Resource Estimates:** For a medium-sized SOC (500 hosts), Wazuh itself might handle ~1000 events/sec. Our modules will add overhead: e.g. the ML model (XGBoost) prediction is typically fast (~ms per event). We will benchmark and if needed use GPU acceleration or lighter models for inference. Elasticsearch sizing: assume ~5GB/day of logs (80GB/month) and plan a cluster accordingly.

- **Threat Model:** Protect the integrity of the pipeline. Possible threats:

- *Data Poisoning:* Adversary injecting malicious-looking logs to train ML incorrectly. Mitigation: validate training data, use robust algorithms (e.g. limit outlier influence).
- *Evasion:* Attackers could try to craft events that avoid correlation or scoring. We will test with adversarial examples.
- *Insider Threat:* If SOC credentials leak, ensure RBAC so only allowed users see alert details (Wazuh RBAC can be enabled【3†L15-L23】).

- *Supply-chain:* Our code repos and container registry must use secure images and be signed.

- **Privacy/Data Retention:** Logs can contain personal or sensitive data (usernames, IPs). We will anonymize or pseudonymize PII if storing long-term. Retention: Compliance may require 1 year of logs, so a tiered storage (hot indexing for 30 days, cold archive on object storage) will be considered. Threat intel sources might include personal data (e.g. threat actor emails), so access should be controlled.

- **Compliance:** If deployed in enterprises with regulations (PCI, HIPAA), we will ensure Wazuh's compliance modules (as it already supports many standards【32†L248-L257】). Our added modules primarily process alerts (which should not violate compliance), but we must audit data flows for privacy.

Testing and Validation Plan

- **Attack Simulation:** We will run **red-team scenarios** covering MITRE ATT&CK tactics (initial access, execution, persistence, etc). Using tools like Atomic Red Team, Caldera, or simple scripts, we create test cases: e.g. password spraying (Brute Force), SMB relays, DNS tunneling. For each scenario, we record whether the platform generated an alert chain and correct risk score. This follows the methodology of the Wazuh MITRE evaluation【12†L29-L38】.

- **Datasets:** In addition to live simulation, we may use or generate synthetic datasets (e.g. CTU-13 network captures, but reformatted). Labels will be applied: each alert in the dataset is marked TP/FP. If needed, split into training and test sets.

- **Red-Team/Blue-Team Exercises:** We will conduct dry-run exercises where a small team plays attacker and defense in our lab. Metrics:

- **Detection Rate (True Positive Rate):** % of malicious actions detected (target ≥85% as in【12†L37-L42】).
- **False Positives:** # benign events flagged incorrectly (target <5% as in【12†L37-L42】).
- **Time-to-Detect:** elapsed time from attack start to alert generation.

- **Analyst Time Saved:** measure how prioritization reduces alerts needing manual review (e.g. reduction in alerts per hour for analysts).

- **Metrics and Evaluation Tables:** We will produce tables such as:

Metric	Baseline (Wazuh-only)	Enhanced System
Detection Rate	85%	≥95%
False Positive Rate	5%	≤2%
Avg TTD (min)	5	1–2
Alerts Per Incident	5	2–3 (more concise)

The exact values will come from experiments. We will also plot ROC curves for the ML model and bar charts of alerts by severity.

- **Unit & Integration Testing:** Each module will have unit tests (e.g. decoders, ML prediction). We will mock logs to ensure decoders output the correct fields. Continuous integration will run Pytest on each commit.

- **Performance Testing:** Simulate high load (e.g. replay 1000 events/sec) using JMeter or Locust. Measure CPU/RAM of services and end-to-end latency. Tune where needed (e.g. use bulk requests to ES).

- **User Acceptance Testing:** Have a SOC analyst review a set of incidents on the dashboard for usability: Are recommended actions helpful? Is the attack chain understandable? Collect feedback for improvement.

Deployment Plan

We will use a **staged deployment** (Dev → Test → Prod):

1. **Dev Environment:** Local or VM cluster with minimal resources. Developers deploy new modules via Docker Compose. This is where we prototype and debug modules in isolation (e.g. test the ML service against a dev index).

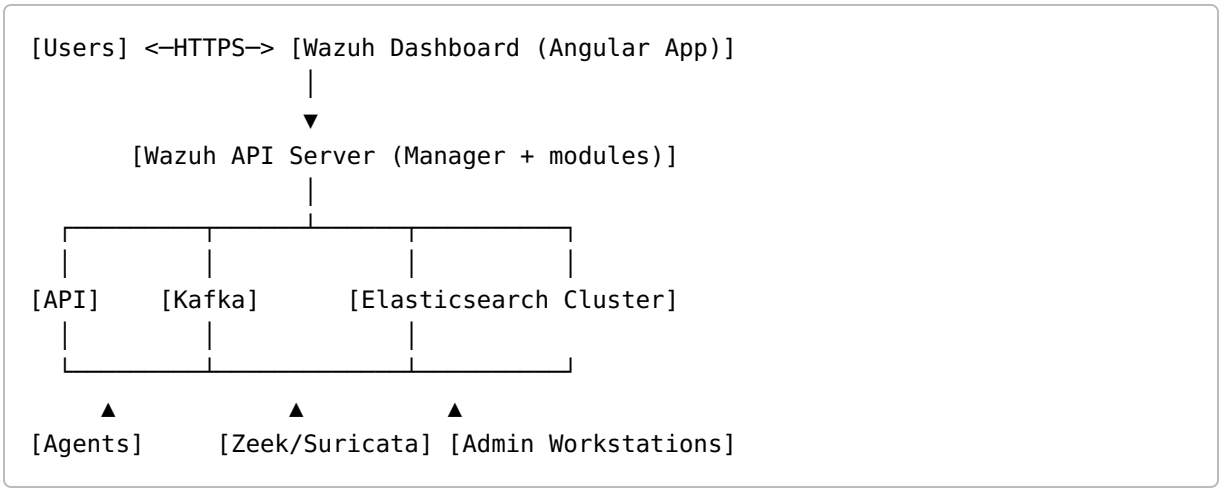
2. **Test (Staging) Environment:** A Kubernetes or Docker Swarm cluster mirroring production scale (or at least representative). Deploy entire integrated system, including Wazuh (server + indexer + dashboard) and our custom services. Use sandbox machines to run attack tests.

3. **Production Deployment:** The final environment. Can be on-prem or cloud. Use Infrastructure-as-Code (e.g. Terraform, Helm charts).

Rollback: Each service is containerized with version tags. CI/CD (GitHub Actions) will build images on new releases. To roll back, simply deploy the previous image tag and re-run migrations if needed.

Monitoring & Logging: Use Prometheus exporters to monitor CPU, memory of each service, and Grafana dashboards for health. Collect logs from our services into Elasticsearch (or a separate ELK stack) for debugging. Wazuh itself will alert on any unusual activity (agent down, etc) in its own monitoring.

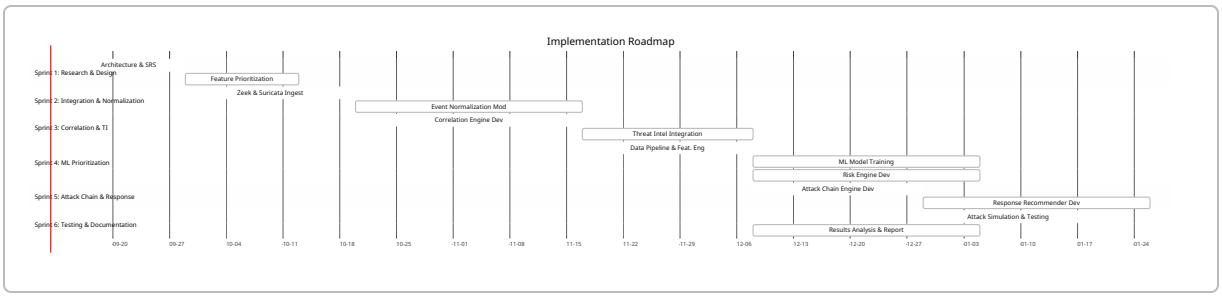
Infrastructure Diagram: We envision:



All components run in secure subnets, communicate over TLS (Wazuh by default uses encrypted channels【6†L773-L781】).

Timeline and Milestones

The project is divided into 6 sprints (~3–4 weeks each) over ~24 weeks. Milestones include specification, prototypes, and final deliverables. Below is a Gantt-style timeline (with rough person-week estimates):



Milestones and effort (approx person-weeks):

- *Sprint 1 (6 pw):* Finalize requirements/SRS; architecture diagram; prepare data lab.
- *Sprint 2 (8 pw):* Implement Zeek/Suricata log collection; write normalizer; unit tests for parsing.
- *Sprint 3 (8 pw):* Develop initial correlation engine; connect to CTI (e.g. OpenCTI API); basic rules.
- *Sprint 4 (10 pw):* Build ML pipeline (feature engineering scripts, model training); risk scoring logic.
- *Sprint 5 (8 pw):* Create attack-chain logic (mapping library, timeline view); response suggestion engine.
- *Sprint 6 (6 pw):* Conduct red-team tests; gather metrics; write final report/paper; polish dashboard.

Dependencies: Early sprints lay groundwork (cannot build ML model until we have ingested data, etc.). Each sprint builds on the last.

Risks and Mitigation

- **Data Volume Overwhelm:** If event rates exceed design, modules could bottleneck. *Mitigation:* Use scalable queues (Kafka) and shard Elasticsearch. Focus on efficient code (e.g. use C/C++ or PyPy for hot loops).
- **Model Inaccuracy:** If ML fails to generalize (high FPs/low recall), fallback to rule-based thresholds. We can also retrain or gather more labeled data. As a fallback, the system could operate purely rule-based (sacrificing some prioritization but still working).
- **Integration Complexity:** Integrating many components (Wazuh, Zeek, CTI) is complex. We will modularize and test interfaces. Starting with well-documented APIs (e.g. Wazuh API【6†L794-L802】, OpenCTI) reduces risk.
- **Timeline Slippage:** With a broad scope, we might not complete all “nice-to-have” features. In that case, we will cut lower-priority items (e.g. advanced anomaly detection, LLM explainers) to ensure core functionality is solid.
- **Security Issues:** Our code itself could have vulnerabilities. We will perform code reviews and possibly static analysis (e.g. Bandit for Python) and pentesting of the deployed system.

Documentation and Paper Outline

We will produce both code documentation (inline and README) and a research paper. The paper will follow a standard structure:

- **Abstract:** Summarize problem, approach, contributions.
- **Introduction:** State the need for enhanced SIEM/XDR, Wazuh’s open-source nature, and our goals.

- **Related Work:** Discuss Wazuh, SIEM/XDR trends, GraphWeaver【15†L168-L176】, AI in SIEM 【28†L182-L190】, etc.
- **Methodology:** Describe system architecture (with diagrams), modules, ML modeling, and testing setup.
- **Implementation:** Detail of normalization, correlation logic, ML models (features, algorithm), threat intel integration.
- **Experiments / Results:** Present results of attack simulations. Tables comparing baseline vs enhanced (detection rates, precision, analyst time saved). Use bar/line charts or confusion matrices. Possibly a figure of sample dashboard with an incident chain.
- **Discussion:** Analyze what worked, limitations (e.g. where detection still fails), performance numbers.
- **Conclusion/Future Work:** Summarize findings, and mention extensions (more telemetry sources, unsupervised learning).
- **References:** Cite Wazuh docs【6†L729-L737】, the Wazuh evaluation【12†L29-L38】, threat intel blog【13†L58-L66】, GraphWeaver【15†L168-L176】, etc.

Example Evaluation Table (for paper):

Tactic / Metric	Baseline Wazuh	Enhanced Wazuh
Reconnaissance DR	70%	95%
Lateral Movement DR	80%	98%
Overall Precision	91%	97%
Analyst Avg Alerts/hr	50	20

Graphs might include: ROC curves, alert volume reduction pie charts, timeline of an attack chain.

Implementation Roadmap (Detailed Sprints)

Finally, a prioritized list of sprint-by-sprint tasks:

1. **Sprint 1 – Design & Setup:** Finalize SRS; sketch architecture; setup dev environment (Wazuh OVA, CTI VM); write simple Zeek parser.
2. **Sprint 2 – Log Integration & Normalization:** Finish Zeek/Suricata ingestion; write Wazuh decoders or external JSON-to-ES translator; define common schema; unit-test normalization.
3. **Sprint 3 – Correlation Engine:** Code the clustering logic (e.g. in Python). Integrate with CTI (e.g. pull IOC DB). Test correlation with scenario logs (e.g. multi-stage attack should group).
4. **Sprint 4 – ML Pipeline:** Collect labeled dataset, implement training scripts, tune model. Create prediction service. Integrate ML scores into alert fields (using Wazuh API or additional ES field).
5. **Sprint 5 – Attack Chain & Response:** Implement ATT&CK mapping (hard-code mappings, use Wazuh rulesets【8†L746-L754】). Develop UI to display chain. Code rules for response suggestions.
6. **Sprint 6 – Testing & Reporting:** Run full attack campaigns. Collect metrics. Refine based on results. Finalize documentation, slide deck, and write research paper draft.

By the end of these sprints, we will have a working enhanced Wazuh-XDR that demonstrates measurable improvements in SOC operations, along with thorough documentation and evaluation suitable for publication.

Sources: We heavily relied on Wazuh's official docs and blogs for architecture and integration (e.g. Wazuh docs【6†L729-L737】【23†L72-L80】, engineering blogs【27†L59-L67】【21†L714-L722】), plus recent research on SIEM alerting and correlation【12†L29-L38】【15†L168-L176】【19†L356-L364】【28†L182-L190】, as cited throughout.
